Secure IoT Device Authentication Using Lightweight Cryptography

(SIOTDA)

A comprehensive project report submitted for the course
Information Security

by

N Ravi Tejesh
(Roll No: CS23B2051)

N Durgalakshmi Prasad
(Roll No: CS23B2052)

Under the guidance of

Dr. Bhale Pradeepkumar Gajendra

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY,
DESIGN AND MANUFACTURING, KANCHEEPURAM

April 22, 2026

List of Figures

14.1 Device Registration and Master Device Selection 35
14.2 Secure Chat Interface from Device 1 36
14.3 Secure Chat Interface from Device 2 36
14.4 Attack Simulation Interface 37
14.5 Cryptography Simulation and Avalanche Effect 37
14.6 Authentication Flow Simulation 38

Contents

1 Introduction **5**
1.1 Background of IoT Security . . . 5
1.2 Why Device Authentication is Critical . . . 5
1.3 Problems in Existing IoT Authentication Systems . . . 6

2 Problem Statement **6**
2.1 Replay Attacks . . . 7
2.2 Device Spoofing . . . 7
2.3 Man-in-the-Middle (MITM) Attacks . . . 7
2.4 Weak IoT Authentication . . . 7

3 Objectives **8**

4 System Architecture **9**
4.1 High-Level Architecture Overview . . . 9
4.2 Frontend Layer . . . 9
4.3 Backend Layer . . . 9
4.4 WebSocket Layer . . . 10
4.5 Database Layer . . . 10

5 Technologies Used **11**
5.1 Python 3 and Flask . . . 11
5.2 Flask-SocketIO and Eventlet . . . 11
5.3 SQLite . . . 11
5.4 JavaScript and Web Crypto API . . . 12
5.5 AES-256-GCM . . . 12
5.5.1 AES-256-GCM Representation . . . 12
5.6 HMAC-SHA256 . . . 12

6 Detailed Module Explanation **13**
6.1 Backend Modules . . . 13
6.1.1 app.py — Application Setup and WebSocket Handling . . . 13
6.1.2 routes.py — REST API Layer . . . 13
6.1.3 database.py — Database Management . . . 14
6.1.4 crypto.py — Cryptographic Operations . . . 14
6.2 Frontend Modules . . . 14
6.2.1 script.js — Core Frontend Logic . . . 14
6.2.2 Client-Side Cryptography . . . 15
6.2.3 WebSocket Communication . . . 15

6.2.4 Chat System . . . 15
6.2.5 Security Dashboard . . . 15
6.2.6 Attack Simulation Module . . . 16
6.2.7 Authentication Flow Visualisation . . . 16

7 API Design **17**
7.1 GET /api/stats . . . 17
7.2 POST /api/register . . . 17
7.3 GET /api/devices . . . 18
7.4 POST /api/auth/flow . . . 18
7.5 POST /api/attack . . . 18
7.6 GET /api/logs . . . 19
7.7 GET /api/messages . . . 19

8 Algorithms Used **20**
8.1 HMAC-SHA256 . . . 20
8.1.1 Mathematical Foundation . . . 20
8.1.2 Message Construction . . . 20
8.1.3 Security Properties . . . 20
8.2 Nonce-Based Replay Prevention . . . 21
8.2.1 Nonce Generation . . . 21
8.2.2 Lifecycle State Machine . . . 21
8.2.3 Replay Prevention Logic . . . 21
8.3 AES-256-GCM Authenticated Encryption . . . 22
8.3.1 Construction Overview . . . 22
8.3.2 Key Parameters . . . 22
8.3.3 Storage Format . . . 22
8.3.4 IV Uniqueness Guarantee . . . 22

9 Attack Simulation **23**
9.1 Replay Attack . . . 23
9.1.1 Attack Scenario . . . 23
9.1.2 Why the Attack Fails . . . 23
9.2 Device Spoofing Attack . . . 23
9.2.1 Attack Scenario . . . 23
9.2.2 Why the Attack Fails . . . 24
9.3 Man-in-the-Middle (MITM) Attack . . . 24
9.3.1 Attack Scenario . . . 24
9.3.2 Why the Attack Fails . . . 24

10 Real-Time Communication **25**
10.1 WebSocket vs. HTTP Polling . . . 25
10.2 Socket.IO Room Architecture . . . 25
10.3 Real-Time Dashboard Updates . . . 26
10.4 Connection State Management . . . 26

11 Database Design **27**
11.1 Schema Overview . . . 27
11.2 Table: devices . . . 27
11.3 Table: nonces . . . 28
11.4 Table: sessions . . . 28
11.5 Table: messages . . . 28
11.6 Table: logs . . . 29
11.7 Additional Notes . . . 29

12 System Workflow **30**
12.1 Device Registration . . . 30
12.2 Authentication Flow . . . 30
12.3 Secure Messaging . . . 31
12.4 Attack Detection . . . 31
12.5 Dashboard Real-Time Update . . . 32

13 Running Procedure **32**
13.1 Prerequisites . . . 32
13.1.1 Step 3: Project Directory Structure . . . 33
13.1.2 Step 4: Start the Backend Server . . . 33
13.1.3 Step 5: Open the Application . . . 33
13.1.4 Step 6: Register a Device . . . 33
13.1.5 Step 7: Register a Second Device and Start Chat . . . 34
13.1.6 Step 8: Run Attack Simulations . . . 34

14 Results and Output **35**
14.1 Device Registration and Master Selection . . . 35
14.2 Chat Section (Device 1) . . . 36
14.3 Chat Section (Device 2) . . . 36
14.4 Attack Simulation . . . 37
14.5 Cryptography Simulation and Avalanche Effect . . . 37
14.6 Authentication Flow Simulation . . . 38
14.7 Backend Execution and Terminal Output . . . 38
14.8 Limitations . . . 39

15 Conclusion **40**

16 References **40**

Chapter 1

Introduction

The rapid growth of interconnected devices has significantly changed how modern systems operate. Instead of isolated components, devices now communicate continuously, enabling automation and intelligent decision-making. However, this connectivity also introduces serious security risks, especially in environments where devices have limited computational capabilities.

At the same time, the rapid expansion of IoT has brought serious security challenges. Many IoT devices operate in open or uncontrolled environments and are built with limited computational power and memory. Due to these constraints, implementing strong security mechanisms becomes difficult, and in many cases, security is either weakened or overlooked. This makes IoT systems more exposed to various types of attacks.

In this project, we focus on improving the security of IoT environments by designing an efficient device authentication mechanism. The aim is to ensure that only verified devices are allowed to communicate, while keeping the solution lightweight so that it can be used even on resource-constrained devices.

1.1 Background of IoT Security

With the increasing number of connected devices, IoT security has become a major concern, especially in applications that handle sensitive data. In practice, many devices are deployed without proper security configurations, often using default credentials or insecure communication methods.

Compared to traditional systems, IoT networks are more complex because they are distributed and consist of heterogeneous devices. This makes centralized security management difficult. Moreover, many devices are designed with priority given to cost and functionality rather than security, which increases the likelihood of vulnerabilities.

1.2 Why Device Authentication is Critical

Ensuring that only legitimate devices participate in communication is essential for maintaining system trust. Without proper verification, unauthorized entities can easily inject false data or disrupt operations. A reliable authentication mechanism helps prevent such threats by validating both identity and message authenticity.

A secure authentication process helps maintain both data integrity and system reliability. It ensures that communication occurs only between trusted devices and reduces

the risk of attacks such as spoofing, replay attacks, and man-in-the-middle attacks.

1.3 Problems in Existing IoT Authentication Systems

Many existing authentication techniques rely on computationally heavy cryptographic methods such as RSA or TLS. While these methods are secure, they are often not suitable for IoT devices with limited resources. As a result, some systems either use simplified security mechanisms or avoid proper authentication altogether.

Common challenges include:

- Use of weak or static credentials that can be easily compromised
- Lack of effective protection against replay and spoofing attacks
- Use of insecure communication channels
- Poor scalability when handling a large number of devices

To overcome these limitations, this project introduces a lightweight authentication approach based on a nonce-driven challenge–response mechanism. This method provides secure device verification while remaining efficient enough for practical IoT deployments.

Chapter 2

Problem Statement

The rapid expansion of IoT systems has brought significant security concerns, particularly in the area of device authentication. A large number of IoT devices operate under strict resource limitations and are often deployed without strong security measures. This makes them easy targets for various types of attacks.

Current authentication techniques present a trade-off: methods that provide strong security are often too resource-intensive for low-power devices, while lightweight approaches tend to be less secure. As a result, unauthorized devices may gain access to the network, which can compromise both data integrity and overall system reliability.

In this project, we aim to design a secure yet lightweight authentication mechanism that addresses these issues while remaining practical for resource-constrained IoT environments.

2.1 Replay Attacks

Replay attacks occur when an attacker intercepts a valid communication and reuses it later to gain unauthorized access. If the system does not track whether a message has already been used, it may accept repeated transmissions as legitimate.

In the absence of protections such as nonce validation or timestamp checks, an attacker can successfully bypass authentication without requiring access to the PSK.

2.2 Device Spoofing

Device spoofing involves an attacker pretending to be a legitimate device by using known identifiers such as device IDs. When authentication relies only on these identifiers, it becomes easy for attackers to impersonate valid devices.

This can lead to the injection of false data or disruption of system operations. To prevent this, authentication must verify not only the identity but also the authenticity of the device through secure methods.

2.3 Man-in-the-Middle (MITM) Attacks

In a man-in-the-middle (MITM) attack, an attacker intercepts communication between two devices and may alter or inject messages without being detected. This type of attack affects both confidentiality and data integrity.

If the system does not include mechanisms to verify message integrity, attackers can manipulate communication and potentially gain unauthorized control.

2.4 Weak IoT Authentication

Due to limited computational resources, many IoT systems rely on simplified or weak authentication mechanisms. While strong protocols such as RSA or TLS provide high security, they are often impractical for low-power devices, leading to the adoption of less secure alternatives.

Common issues include:

- Use of weak or static credentials that are easy to exploit
- Insufficient protection against replay and spoofing attacks
- Lack of proper mechanisms to ensure message integrity

These limitations emphasize the need for an authentication approach that balances security and efficiency, making it suitable for real-world IoT deployments.

Chapter 3

Objectives

The primary objective of this project is to design and implement a secure and efficient authentication system for IoT devices using lightweight cryptographic techniques. The system is intended to allow only authorized devices to access the network while keeping computational requirements low enough for resource-constrained environments.

The specific objectives of this project are as follows:

- To develop a lightweight authentication mechanism that can be effectively deployed on IoT devices with limited processing power and memory.
- To design a secure challenge–response protocol based on nonce verification, ensuring that each authentication session is unique and cannot be reused.
- To protect the system against common threats such as replay attacks, device spoofing, and man-in-the-middle (MITM) attacks through appropriate validation and cryptographic methods.
- To use HMAC-SHA256 for device authentication in a way that avoids transmitting sensitive information such as PSKs over the network.
- To enable secure communication between devices using AES-256-GCM encryption, ensuring both confidentiality and integrity of the exchanged data.
- To implement a complete system that includes backend, frontend, and database components, demonstrating real-time authentication, secure messaging, and attack detection.
- To design an interactive interface that helps visualize the authentication process, encryption operations, and attack simulations for better understanding.
- To ensure that the system can scale to support multiple devices while maintaining performance and security.

The overall aim is to build a practical and reliable solution that achieves a balance between security and efficiency, making it suitable for real-world IoT applications.

Chapter 4

System Architecture

4.1 High-Level Architecture Overview

The SIOTDA system is designed using a three-tier architecture that includes a frontend (presentation layer), backend (application layer), and database (persistence layer). Each layer is responsible for a specific part of the system, which helps maintain both efficiency and security.

The frontend handles user interaction, the backend manages application logic and security-related operations, and the database is used to store system data. In addition to these layers, a WebSocket component is used to enable real-time communication between the frontend and backend.

The system is intended for local deployment, making it lightweight and easy to set up without requiring external services.

4.2 Frontend Layer

The frontend is implemented using HTML, CSS, and JavaScript. It consists of multiple pages such as registration, chat, dashboard, authentication flow visualization, and attack simulation.

Communication with the backend is carried out using:

- **REST API:** Used for operations like device registration, authentication flow, and fetching data
- **WebSocket:** Used for real-time updates such as messaging and dashboard changes

Cryptographic operations on the client side are handled using the Web Crypto API, which allows secure encryption and hashing directly within the browser.

4.3 Backend Layer

The backend is developed using Flask and is structured into separate modules for better organization and maintainability:

- **app.py:** Initializes the application and manages WebSocket communication
- **routes.py:** Defines REST API endpoints

- **crypto.py:** Handles cryptographic functions
- **database.py:** Manages database operations

Flask-SocketIO, along with eventlet, is used to support asynchronous communication. This allows the system to handle multiple real-time interactions without blocking API requests.

4.4 WebSocket Layer

The WebSocket layer is responsible for enabling real-time communication between clients and the server. Each connected device is assigned to a specific room, ensuring that messages are delivered only to the intended recipient.

Some of the key events handled include:

- connect and disconnect
- join
- send_message and receive_message
- devices_update and log_update

This approach reduces the need for repeated HTTP requests and allows faster communication between components.

4.5 Database Layer

The system uses SQLite as the database, which provides a simple and lightweight storage solution. To improve performance during concurrent operations, Write-Ahead Logging (WAL) mode is enabled.

The primary tables used in the system are:

- devices
- nonces
- sessions
- messages
- logs

This database design supports secure storage of important information such as device credentials, authentication data, communication history, and system logs.

Chapter 5

Technologies Used

5.1 Python 3 and Flask

The backend of the system is implemented using Python 3 along with the Flask framework. Flask is a lightweight web framework that provides core features such as URL routing, request handling, and JSON response support, making it suitable for building efficient APIs.

In this project, REST API endpoints are structured using Blueprints defined in *routes.py* and registered in *app.py*. This modular approach helps keep the code organized and easier to maintain. Flask also allows direct interaction with the database using Python's built-in sqlite3 module, avoiding the need for additional tools such as ORM frameworks.

5.2 Flask-SocketIO and Eventlet

For real-time communication, Flask-SocketIO is used to establish WebSocket connections between the client and server. This enables continuous data exchange without the need for repeated HTTP requests.

Socket.IO provides useful features such as automatic reconnection and room-based communication. In this system, each device is assigned to a specific room so that messages are delivered only to the intended recipient.

Eventlet is used on the backend to support asynchronous processing. It utilizes lightweight green threads, allowing the system to handle multiple connections efficiently without blocking operations.

5.3 SQLite

SQLite is used as the database for this project. It is a lightweight and serverless database that stores all data in a single file, making it simple to set up and manage.

Database operations are performed using Python's built-in sqlite3 module, which eliminates external dependencies while providing full control over queries.

To improve performance, Write-Ahead Logging (WAL) mode is enabled. This allows read operations to continue even when write operations are in progress, which is particularly useful for real-time applications.

5.4 JavaScript and Web Crypto API

The frontend is developed using plain JavaScript, keeping the implementation simple and lightweight.

All cryptographic operations on the client side are handled using the Web Crypto API, which is available in modern browsers and provides secure implementations of common cryptographic functions.

The main client-side operations include:

- Generating SHA-256 hashes for key derivation
- Importing keys for encryption and decryption
- Encrypting and decrypting messages using AES-256-GCM
- Generating secure random values for initialization vectors (IVs)

This approach ensures that cryptographic processing is handled securely without relying on external libraries.

5.5 AES-256-GCM

AES-256-GCM is used to provide secure message encryption, ensuring both confidentiality and data integrity.

Each message is encrypted using a 256-bit key along with a randomly generated initialization vector (IV), producing ciphertext along with an authentication tag. Any modification to the encrypted data causes the decryption process to fail.

In this system, the encryption key is derived using SHA-256, and a new random IV is generated for every message to maintain security.

5.5.1 AES-256-GCM Representation

AES-256-GCM performs authenticated encryption as:

$$C,\ T = \text{AES-GCM}_K(IV,\ P)$$

5.6 HMAC-SHA256

HMAC-SHA256 is used for device authentication by generating a message authentication code using a shared PSK.

The device computes an HMAC value based on the challenge received from the server, and the server verifies it by performing the same computation. If both results match, the device is considered authenticated.

This method ensures that the PSK is never transmitted over the network.

Chapter 6

Detailed Module Explanation

6.1 Backend Modules

6.1.1 app.py — Application Setup and WebSocket Handling

The `app.py` file acts as the central controller of the backend. It initializes the application, connects different modules, and manages real-time communication between clients using WebSocket events.

It sets up the Flask app, enables CORS for API access, and integrates Socket.IO to support real-time interactions. The REST API routes are also registered here, connecting all backend functionalities to the main application.

In addition, this file includes helper functions that broadcast updates, such as device lists and system logs, to all connected clients. This allows the frontend to reflect changes instantly without requiring a page refresh.

It also manages WebSocket events, including client connections, disconnections, joining rooms, and message exchange between devices.

6.1.2 routes.py — REST API Layer

The file `routes.py` defines all REST API endpoints used by the frontend. These endpoints are organized using a Blueprint structure, which helps keep the code modular and easy to maintain.

The APIs handle:

- Device registration
- Device listing
- Authentication flow simulation
- Attack simulation
- Log retrieval
- Statistics collection

Each endpoint processes input received from the frontend, interacts with the database when required, and returns structured JSON responses.

The authentication flow API demonstrates the complete challenge–response process, while the attack API is used to simulate and detect different types of attacks.

6.1.3 database.py — Database Management

The file `database.py` is responsible for handling all database-related operations using SQLite.

Its main responsibilities include:

- Creating required tables during initialization
- Managing database connections
- Executing queries for storing and retrieving data

The database stores device information, authentication nonces, session tokens, encrypted messages, and system logs. This ensures that all activities within the system are properly recorded and can be accessed when needed.

6.1.4 crypto.py — Cryptographic Operations

The file `crypto.py` handles all cryptographic functionalities required by the system.

Its responsibilities include:

- Generating HMAC values for authentication
- Verifying received HMAC values
- Generating secure nonces
- Creating session tokens
- Performing AES-256-GCM encryption and decryption

These operations ensure that authentication and communication remain secure without exposing any sensitive information such as PSKs.

6.2 Frontend Modules

6.2.1 script.js — Core Frontend Logic

The file `script.js` contains the main logic of the frontend. It manages user interactions, communicates with the backend, and updates the user interface dynamically.

Its responsibilities include:

- Making API calls to the backend
- Handling WebSocket communication
- Managing page initialization and events
- Rendering data on UI components

All frontend pages rely on this single script file, which simplifies maintenance and keeps the implementation consistent.

6.2.2 Client-Side Cryptography

The frontend performs encryption and decryption using the browser's Web Crypto API.

It is responsible for:

- Generating encryption keys from shared secrets
- Encrypting messages before sending them
- Decrypting messages after receiving them

This ensures that messages remain protected even before they are transmitted to the server.

6.2.3 WebSocket Communication

The frontend establishes a WebSocket connection with the backend using Socket.IO, enabling continuous communication between the client and server.

This supports:

- Real-time message exchange
- Immediate updates of device lists
- Live dashboard updates
- Event-driven communication

Since the connection remains active, it avoids the need for repeated HTTP requests.

6.2.4 Chat System

The chat module enables secure communication between devices.

It includes:

- Device selection interface
- Message input and display
- Encryption before sending messages
- Decryption after receiving messages

It also provides an option to simulate MITM attacks for demonstration and testing purposes.

6.2.5 Security Dashboard

The dashboard provides a real-time view of system activity.

It displays:

- Registered devices
- Authentication results

- Detected attacks
- System logs

The data is updated automatically whenever new events occur, ensuring that the information remains current.

6.2.6 Attack Simulation Module

This module allows users to simulate different types of attacks such as replay attacks, spoofing, and MITM.

It helps users:

- Understand how these attacks are performed
- Observe how the system detects and blocks them
- Visualize the effectiveness of implemented security mechanisms

6.2.7 Authentication Flow Visualisation

This module presents a step-by-step view of the authentication process.

It includes:

- Device request initiation
- Nonce generation
- HMAC computation
- Verification process
- Session creation

This visual representation makes it easier to understand how the authentication mechanism works in practice.

Chapter 7

API Design

The backend provides a set of REST API endpoints that manage communication between the frontend and the server. These APIs are designed to be simple, consistent, and JSON-based, making them easy to use, test, and integrate.

Each endpoint serves a specific purpose, such as device registration, authentication, data retrieval, or attack simulation. Standard HTTP methods like GET and POST are used throughout the system.

7.1 GET /api/stats

This endpoint retrieves system statistics required for the dashboard. It returns aggregated information such as the total number of devices, messages exchanged, and detected security events.

Purpose: Display real-time system status on the dashboard.

Response:

```
{
  "devices": 5,
  "messages": 120,
  "attacks_detected": 3
}
```

7.2 POST /api/register

This API handles the onboarding of new devices into the system. It accepts a unique device identifier along with a pre-shared key and securely stores this information for future authentication processes.

Purpose: Add a new device to the system.

Request:

```
{
  "deviceId": "device_1",
  "secretKey": "my_secret_key"
}
```

Response:

```
{
  "success": true,
```

```
  "device_id": "device_1"
}
```

7.3 GET /api/devices

This endpoint returns the list of all registered devices. It is mainly used to populate device lists and selection options in the frontend.

Purpose: Retrieve all registered devices.

Response:

```
{
  "devices": ["device_1", "device_2"]
}
```

7.4 POST /api/auth/flow

This endpoint simulates the complete authentication process using a challenge–response mechanism. It generates a nonce, computes an HMAC value, and verifies the device.

It also supports different modes to demonstrate system behavior:

- normal
- wrong_psk
- replay

Purpose: Demonstrate the authentication process step-by-step.

Request:

```
{
  "device_id": "device_1",
  "mode": "normal"
}
```

Response (simplified):

```
{
  "status": "success",
  "nonce": "abc123",
  "hmac": "xyz456"
}
```

7.5 POST /api/attack

This endpoint is used to simulate different attack scenarios such as replay, spoofing, and MITM. It executes the selected attack and returns the corresponding result.

Purpose: Demonstrate how attacks are detected and handled by the system.

Request:

```
{
  "type": "replay",
  "device": "device_1"
}
```

Response:

```
{
  "result": "Replay attack detected and blocked"
}
```

7.6 GET /api/logs

This endpoint retrieves all system logs, including authentication attempts and detected attacks. It is used by the dashboard to display security-related events.

Purpose: Monitor system activity and security events.

Response:

```
[
  {
    "event": "AUTH",
    "device": "device_1",
    "description": "Authentication successful"
  }
]
```

7.7 GET /api/messages

This endpoint returns stored messages exchanged between devices. The messages are encrypted and include metadata such as sender, receiver, and timestamp.

Purpose: Retrieve communication history.

Response:

```
[
  {
    "sender": "device_1",
    "receiver": "device_2",
    "ciphertext": "encrypted_data",
    "timestamp": "2026-04-22"
  }
]
```

Chapter 8

Algorithms Used

This chapter describes the main algorithms used in the system for authentication, replay protection, and secure communication. The focus is on using cryptographic techniques that are both secure and efficient enough for IoT environments.

8.1 HMAC-SHA256

HMAC-SHA256 is used for device authentication. It ensures that only devices with the correct PSK are able to generate a valid response.

8.1.1 Mathematical Foundation

HMAC (Hash-based Message Authentication Code) is defined in RFC 2104. Given a hash function H:

$$HMAC(K, M) = H\big((K_0 \oplus opad) \parallel H\big((K_0 \oplus ipad) \parallel M\big)\big)$$

For SHA-256:

- Block size $B = 64$ bytes
- Output size $L = 32$ bytes

8.1.2 Message Construction

In this system, the message is constructed as:

$$M = device_id \parallel timestamp \parallel nonce \parallel payload$$

The same message format is used on both the backend and frontend, ensuring consistent HMAC computation.

8.1.3 Security Properties

- **Authentication:** Confirms the identity of the device using a shared PSK
- **Integrity:** Ensures that the message has not been altered
- **Key Protection:** The PSK is never transmitted over the network

8.2 Nonce-Based Replay Prevention

Replay attacks are mitigated by using a unique nonce for each authentication request.

8.2.1 Nonce Generation

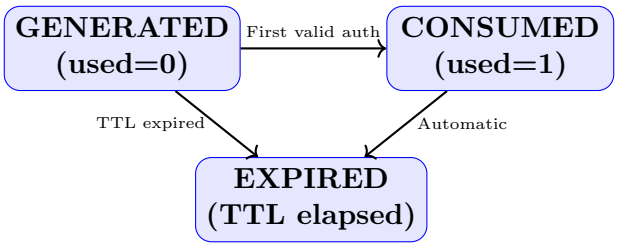
A nonce is generated using a secure random function:

$$\text{nonce} = \text{SecureRandom}(128\text{-bit})$$

This guarantees both unpredictability and uniqueness.

8.2.2 Lifecycle State Machine

Each nonce follows a simple lifecycle:



- Generated
- Issued to the device
- Used during authentication
- Expired after use or timeout

8.2.3 Replay Prevention Logic

Before accepting a request, the system verifies:

- Whether the nonce exists
- Whether it has already been used
- Whether it has expired

If any of these checks fail, the request is rejected.

Listing 8.1: Nonce validation logic

```
def is_nonce_valid(nonce):
    row = get_nonce(nonce)

    if not row:
        return False
    if row.used:
        return False
    if current_time() > row.expiry:
```

```
        return False

    return True
```

8.3 AES-256-GCM Authenticated Encryption

AES-256-GCM is used to secure communication between devices. It provides both encryption and integrity protection.

8.3.1 Construction Overview

AES-256-GCM combines encryption and authentication:

$$C, T = \text{AES-GCM}_K(IV, P)$$

where:

- P = plaintext
- C = ciphertext
- T = authentication tag

8.3.2 Key Parameters

Parameter	Value	Description
Key size	256 bits	Encryption key
IV size	96 bits	Random per message
Tag size	128 bits	Integrity check

8.3.3 Storage Format

Encrypted messages are stored in the following format:

$$IV : Ciphertext : Tag$$

This ensures that all required components are available for decryption.

8.3.4 IV Uniqueness Guarantee

A new random IV is generated for each encryption operation. This ensures that even if the same message is sent multiple times, the resulting ciphertext will be different, preventing pattern-based attacks.

Chapter 9

Attack Simulation

This chapter demonstrates how common network attacks are simulated within the system and explains how the proposed authentication mechanism handles and prevents them.

9.1 Replay Attack

9.1.1 Attack Scenario

A replay attack occurs when an attacker captures a valid authentication message and attempts to reuse it later to gain unauthorized access.

The sequence of actions is as follows:

- The server generates and sends a nonce to the device.
- The device computes an HMAC using the nonce and returns it.
- The attacker intercepts this valid request.
- At a later time, the attacker resends the same request to the server.

9.1.2 Why the Attack Fails

The system prevents replay attacks using strict nonce validation.

- Each nonce is marked as **used** after a successful authentication.
- If the same nonce appears again, the request is immediately rejected.
- Nonces also have a limited lifetime (TTL), after which they become invalid.

Because of these checks, even a valid captured message cannot be reused. The system identifies that the nonce is no longer valid and blocks the request.

9.2 Device Spoofing Attack

9.2.1 Attack Scenario

In a device spoofing attack, an attacker attempts to impersonate a legitimate device by using its identifier.

- The attacker obtains or guesses a valid device ID.
- An authentication request is sent using that identity.
- The server responds with a nonce as part of the challenge.
- The attacker tries to generate a valid response without the PSK.

9.2.2 Why the Attack Fails

The system relies on HMAC-based authentication, which requires the correct PSK.

- The attacker does not possess the pre-shared key (PSK).
- Any HMAC generated without the correct key will be invalid.
- The server verifies the received HMAC by computing its own version.

Since the computed values do not match, the authentication attempt fails and access is denied.

9.3 Man-in-the-Middle (MITM) Attack

9.3.1 Attack Scenario

In a man-in-the-middle (MITM) attack, an attacker intercepts communication between devices and attempts to modify or inject data.

- The attacker intercepts a message exchanged between two devices.
- The message is altered or replaced with malicious content.
- The modified message is forwarded to the intended receiver.

9.3.2 Why the Attack Fails

The system uses AES-256-GCM encryption, which ensures both confidentiality and integrity.

- Each encrypted message includes an authentication tag.
- Any modification to the message results in tag verification failure.
- The system automatically rejects such tampered messages.

As a result, the attacker cannot alter or inject data without being detected, and the communication remains secure.

Chapter 10

Real-Time Communication

10.1 WebSocket vs. HTTP Polling

Traditional web applications often retrieve updates by periodically polling the server (for example, sending a GET request to `/api/logs` every few seconds). While simple, this approach introduces several limitations in a security monitoring system:

- **Latency:** If a security event occurs just after a polling cycle, it may take several seconds before it becomes visible to the user.
- **Bandwidth overhead:** Polling generates repeated HTTP requests, including headers and handshakes, even when there is no new data.
- **Server load:** A large number of clients continuously polling the server can create unnecessary load, regardless of actual activity.

WebSocket addresses these issues by establishing a persistent connection after the initial HTTP upgrade. Once connected, the server can push updates to clients immediately when new data becomes available, resulting in faster and more efficient communication.

10.2 Socket.IO Room Architecture

Flask-SocketIO provides a room-based mechanism for targeted message delivery. When a user selects a device, the frontend emits a `join` event, which triggers the server to call `join_room(device_id)`. This associates the client's socket with a specific room.

When a message is sent from one device to another, the server emits events only to the relevant rooms:

```
socketio.emit("receive_message", msg, room="Device_B")
socketio.emit("receive_message", msg, room="Device_A")
```

As a result, only the intended devices receive the message. Other connected clients do not receive any unnecessary data, ensuring efficient communication and maintaining privacy at the delivery layer (in addition to AES-256-GCM encryption at the content level).

10.3 Real-Time Dashboard Updates

The dashboard listens for the `log_update` WebSocket event to reflect system changes in real time:

```
sock.on("log_update", () => {
  if (document.getElementById("logTableBody")) loadDashboard();
});
```

Whenever a security-related event occurs (such as authentication, attack detection, or device registration), the backend emits a `log_update` event. This is broadcast to all connected clients.

On receiving this event, the dashboard fetches updated statistics and logs through REST APIs and refreshes the UI almost instantly.

10.4 Connection State Management

The `connection-bar` element at the top of each page displays the current WebSocket connection status. Initially, it shows "Connecting...", then changes to "Connected" (green) when the Socket.IO `connect` event is triggered. If the connection is lost or an error occurs, it switches to "Offline" (red).

While the system is offline, all interactive actions (such as sending messages or running simulations) are disabled. This prevents the user from performing operations when the backend is not available.

Chapter 11

Database Design

This chapter describes how data is organized and stored within the system. SQLite is used as the database, as it is lightweight, file-based, and does not require a separate server. All data is stored in a single file named `iot_security.db`.

The database is initialized during application startup using the `init_db()` function, which ensures that all required tables are created if they do not already exist.

11.1 Schema Overview

The database consists of five main tables:

- devices
- nonces
- sessions
- messages
- logs

Each table is responsible for handling a specific aspect of the system, such as authentication, communication, or security monitoring.

11.2 Table: devices

The `devices` table stores information about all registered IoT devices.

- **device_id:** Unique identifier for each device (Primary Key)
- **psk:** Pre-shared key used for authentication
- **registered_at:** Timestamp indicating when the device was registered

This table is referenced during authentication to verify device identity and retrieve the corresponding PSK.

11.3 Table: nonces

The `nonces` table stores the challenge values used during the authentication process.

- **nonce:** Unique random value (Primary Key)
- **device_id:** Associated device
- **created_at:** Time when the nonce was generated
- **used:** Indicates whether the nonce has already been used (0 or 1)
- **ttl:** Time-to-live value that determines expiration

A foreign key constraint links `device_id` to the `devices` table. This table plays an important role in preventing replay attacks.

11.4 Table: sessions

The `sessions` table stores session tokens issued after successful authentication.

- **token:** Unique session token (Primary Key)
- **device_id:** Associated device
- **created_at:** Time when the session was created
- **expires_at:** Time when the session expires

This table helps maintain active sessions and ensures that expired sessions cannot be reused.

11.5 Table: messages

The `messages` table stores encrypted messages exchanged between devices.

- **id:** Auto-incremented message ID (Primary Key)
- **sender:** Sender device ID
- **receiver:** Receiver device ID
- **ciphertext:** Encrypted message content
- **iv:** Initialization vector used during encryption
- **timestamp:** Time when the message was sent

All messages are stored in encrypted form to maintain confidentiality.

11.6 Table: logs

The `logs` table records all system events and security-related activities.

- **id:** Auto-incremented log ID (Primary Key)
- **timestamp:** Time of the event
- **event:** Type of event (e.g., AUTH, REPLAY, MITM)
- **device:** Related device
- **description:** Detailed information about the event

This table is useful for monitoring system activity, debugging issues, and analyzing security behavior.

11.7 Additional Notes

The database operates in Write-Ahead Logging (WAL) mode, which allows read operations to continue while write operations are in progress. This improves performance in real-time scenarios where frequent reads and writes occur.

All database interactions are performed using Python's built-in `sqlite3` module with row-based access, making it easier to retrieve column values by name.

Chapter 12

System Workflow

This chapter outlines how the system operates step by step, starting from device registration and extending to real-time updates. It describes how different components interact during both normal operation and security-related scenarios.

12.1 Device Registration

1. The user opens the `register.html` page.
2. A Device ID (e.g., `SENSOR_001`) and a Pre-Shared Key are entered.
3. The frontend validates the input (such as minimum length and allowed characters).
4. A request is sent to `POST /api/register`.
5. The backend verifies the input and checks whether the device already exists.
6. If valid, the device is stored in the `devices` table.
7. A `REGISTER` event is recorded in the system logs.
8. The server emits a `devices_update` event via WebSocket.
9. All connected clients update their device lists automatically.
10. The user selects the registered device as the master device.

12.2 Authentication Flow

1. The user selects a device and initiates the authentication process.
2. The frontend sends a request to `POST /api/auth/flow`.
3. The backend retrieves the corresponding PSK from the database.
4. A secure random nonce is generated and stored.
5. An HMAC is computed using the device data and nonce.
6. In normal conditions, the nonce is consumed and authentication succeeds.

7. In invalid scenarios (such as incorrect key or replay), authentication fails.
8. The result is logged in the system.
9. A `log_update` event is broadcast to all connected clients.
10. The frontend displays the authentication process step by step.

12.3 Secure Messaging

1. Both sender and receiver devices must be registered.
2. The user opens `chat.html` and selects a target device.
3. A message is entered and sent.
4. The frontend derives an AES-256 key using SHA-256.
5. A random initialization vector (IV) is generated.
6. The message is encrypted using AES-256-GCM.
7. The encrypted data is transmitted via WebSocket.
8. The backend stores the encrypted message in the database.
9. The message is delivered to the intended recipient.
10. The receiver decrypts and displays the message.

12.4 Attack Detection

1. The user opens `attack.html`.
2. A device and attack type are selected.
3. The frontend sends a request to `POST /api/attack`.
4. The backend simulates the selected attack scenario.
5. If the attack is detected, it is blocked and logged.
6. A `log_update` event is broadcast to all clients.
7. The frontend displays the result in a terminal-style output.
8. If the dashboard is open, it updates automatically.

12.5 Dashboard Real-Time Update

1. The user opens `dashboard.html`.
2. Initial data is loaded from `/api/stats` and `/api/logs`.
3. The data is displayed using charts and tables.
4. A WebSocket listener is activated for live updates.
5. Whenever an event occurs (authentication, attack, or registration), the server emits `log_update`.
6. The dashboard receives this event and refreshes automatically.
7. Updated information is displayed instantly without requiring a page reload.

Chapter 13

Running Procedure

This chapter describes the steps required to set up and run the project.

13.1 Prerequisites

Requirements:

- Python 3.9 or later
- Required libraries: flask, flask-socketio, flask-cors, eventlet, cryptography
- A web browser (Chrome, Firefox, Edge, or Safari)

Installation Steps:

- Clone or download the project repository from GitHub:
  - **GitHub Repository Link**
- Navigate to the backend directory:
  - **cd backend**
- Install the required dependencies:
  - **pip install flask flask-socketio flask-cors eventlet cryptography**

The main dependencies include Flask for backend development, Socket.IO for real-time communication, Eventlet for asynchronous processing, and Cryptography for secure operations.

13.1.1 Step 3: Project Directory Structure

Ensure that the project is organized as follows:

```
siotda/
  backend/
    app.py
    routes.py
    database.py
    utils/
      __init__.py
      crypto.py
  frontend/
    index.html
    register.html
    chat.html
    dashboard.html
    attack.html
    flow.html
    crypto.html
    encryption.html
    script.js
    styles.css
```

13.1.2 Step 4: Start the Backend Server

- python app.py Frontend

After running the above command, the server will start at:

- http://localhost:5000

13.1.3 Step 5: Open the Application

Open a web browser and navigate to:

- http://localhost:5000

Within a few seconds, the connection status should display **Connected**.

13.1.4 Step 6: Register a Device

1. Open the Register page.
2. Enter a Device ID and PSK.
3. Click **Register Device**.
4. Select the device as the master device.

13.1.5 Step 7: Register a Second Device and Start Chat

1. Open another browser tab.
2. Register a second device using the same PSK.
3. Set it as the master device.
4. Open the Chat page in both tabs.
5. Send messages between the two devices.
6. The messages will be encrypted and decrypted automatically.

13.1.6 Step 8: Run Attack Simulations

1. Open the Attack page.
2. Select a device.
3. Choose an attack type (Replay, Spoof, MITM).
4. Run the simulation.
5. View the results in the output panel.
6. Monitor the dashboard for real-time logs.

Chapter 14

Results and Output

This chapter presents the outputs of the system along with screenshots of different modules. Each section highlights how the system behaves in real-time and demonstrates the implemented security features.

14.1 Device Registration and Master Selection

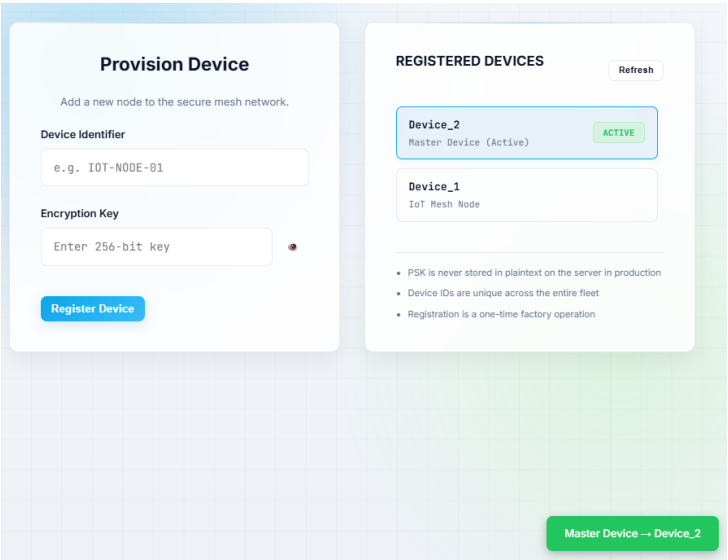


Figure 14.1: Device Registration and Master Device Selection

This interface allows users to register a new device by entering a device ID and a PSK. Once registered, the device appears in the available device list. The user can then select one device as the master device, which is used for further operations.

14.2 Chat Section (Device 1)

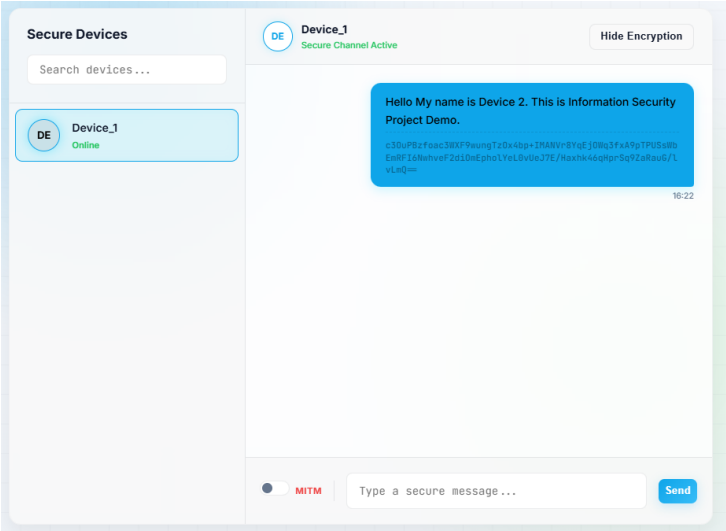


Figure 14.2: Secure Chat Interface from Device 1

This screenshot shows the chat interface from the perspective of Device 1. Messages are encrypted before being sent, ensuring secure communication. The interface also displays connection status and active device interaction.

14.3 Chat Section (Device 2)

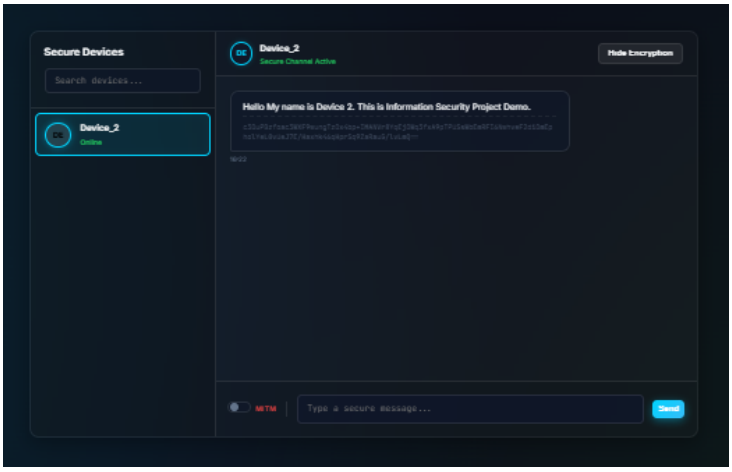


Figure 14.3: Secure Chat Interface from Device 2

This view represents the chat interface from Device 2. Incoming messages are decrypted automatically, demonstrating secure end-to-end communication between devices.

14.4 Attack Simulation

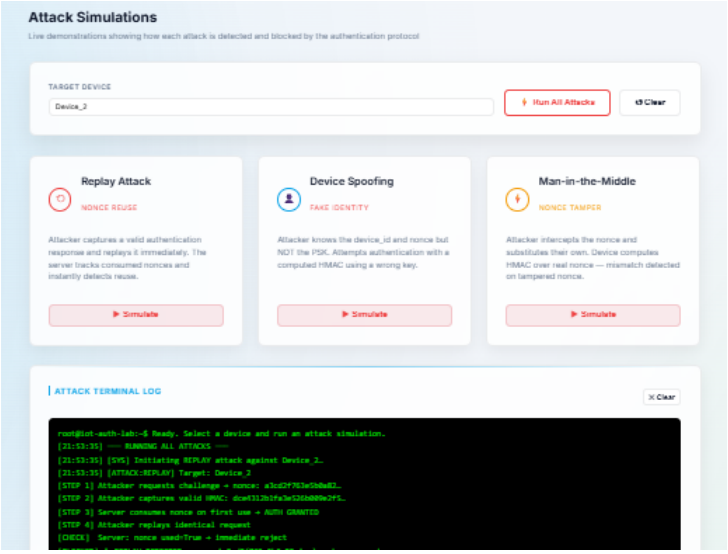


Figure 14.4: Attack Simulation Interface

This module demonstrates how attacks such as replay, spoofing, and MITM are simulated within the system. The system detects these attacks and blocks them accordingly. The output panel provides step-by-step logs, showing how each attack is handled.

14.5 Cryptography Simulation and Avalanche Effect

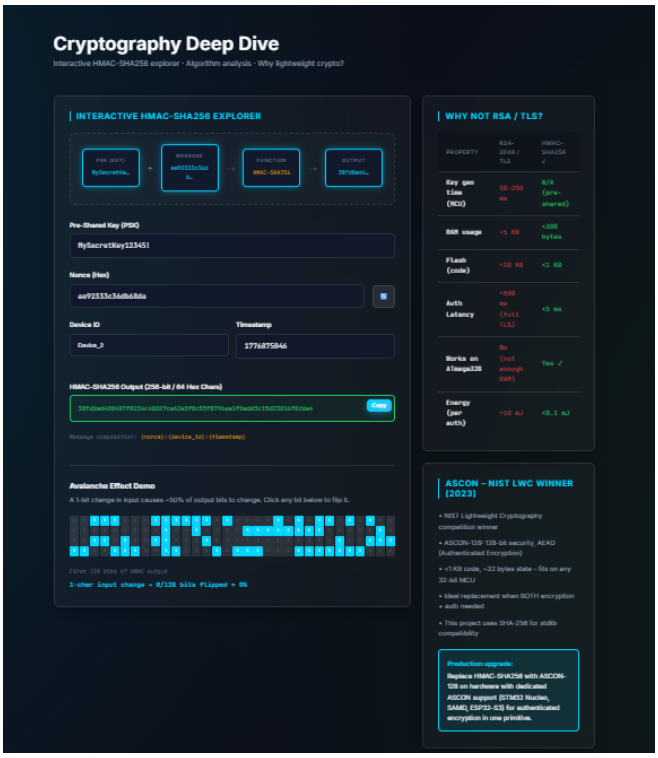


Figure 14.5: Cryptography Simulation and Avalanche Effect

This module demonstrates HMAC-SHA256 and the avalanche effect, where small input changes produce significantly different outputs, highlighting strong security.

14.6 Authentication Flow Simulation

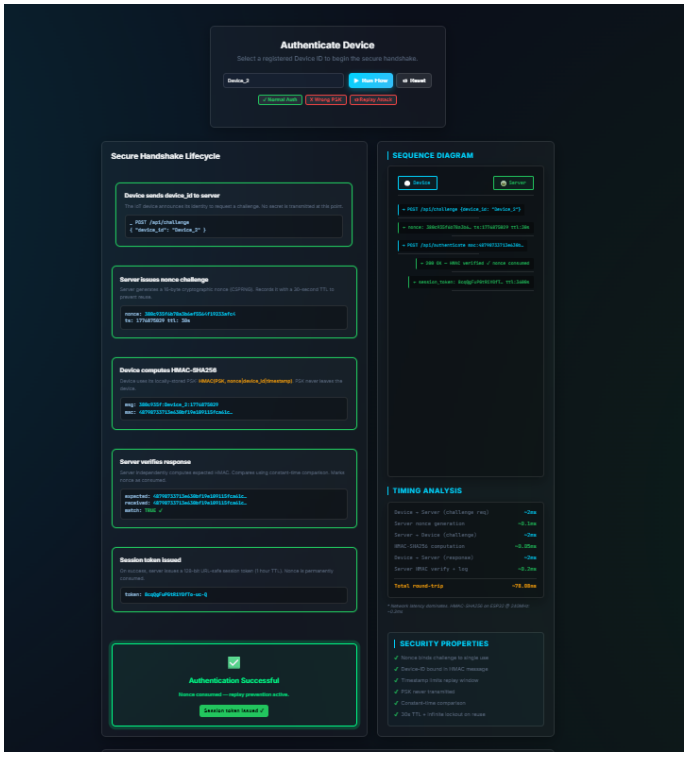


Figure 14.6: Authentication Flow Simulation

This section presents the authentication process, including nonce generation, HMAC computation, verification, and session creation. The diagram illustrates how secure communication is established.

14.7 Backend Execution and Terminal Output

Listing 14.1: Backend Server Execution and Real-Time Logs

```
PS C:\Users\ravit\Downloads\Projects\Information_Security>
   python Backend/app.py Frontend

[DB]  SQLite initialised -> iot_security.db
[INFO] Serving frontend from: ...\Frontend

============================================================
  SIOTDA -- Real-Time WebSocket Edition
  Server  ->  http://localhost:5000
  Stop    ->  Ctrl+C
============================================================
```

```
[WS] Client connected
[JOIN] Socket joined room: Device_2

============================================================
[BACKEND RECEIVED]
  Sender   : Device_2
  Receiver : device_1
  MITM     : False
  IV       : N9IOVPubHE/tW39x
  Data     : c3OuPBzfoac3WXF9...
============================================================

[DELIVERED] Device_2 -> Device_1

============================================================
[BACKEND RECEIVED]
  Sender   : Device_2
  Receiver : Device_1
  MITM     : True
  IV       : dwPrah4ki1lMFtux
  Data     : TGJfaLDxxpqXxoQi...
============================================================
```

The terminal output above shows how the backend operates in real time. It includes:

- Server initialization and database setup
- WebSocket client connections and room assignment
- Transmission of encrypted messages between devices
- Detection of MITM attacks when simulation is enabled

These logs clearly demonstrate how messages are processed, delivered, and blocked in case of malicious activity.

14.8 Limitations

- **Pre-Shared Key Dependency:** The system relies on manually shared PSKs, which may not scale well in larger IoT deployments.
- **Limited Scalability:** SQLite is not ideal for large-scale systems with high concurrency.
- **Basic MITM Simulation:** The MITM attack is simulated at the application level rather than fully implemented at the network level.
- **No Key Management:** The system does not currently support key rotation or secure key distribution.

Chapter 15

Conclusion

In this project, a secure and efficient authentication system for IoT devices has been designed and implemented using lightweight cryptographic techniques. The approach demonstrates that strong security can be achieved even in resource-constrained environments by carefully selecting suitable algorithms such as HMAC-SHA256 and AES-256-GCM.

- A nonce-based challenge–response authentication mechanism was successfully implemented to ensure secure device verification.
- Common attacks such as replay, spoofing, and man-in-the-middle (MITM) were effectively detected and prevented.
- Real-time communication was achieved using WebSocket, enabling immediate updates and interaction between devices.
- Data confidentiality and integrity were maintained using AES-256-GCM encryption.

Overall, this project shows that lightweight cryptographic methods can provide a practical balance between security and performance.

Chapter 16

References

- H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, 1997.
- D. He, N. Kumar, J. Chen, and S. Lee, "Secure Authentication Scheme for IoT Applications," IEEE Internet of Things Journal, 2017.
- Research and Improvement of AES Encryption Algorithm Based on Three-Dimensional Chaotic Mapping, IEEE Xplore, 2026.